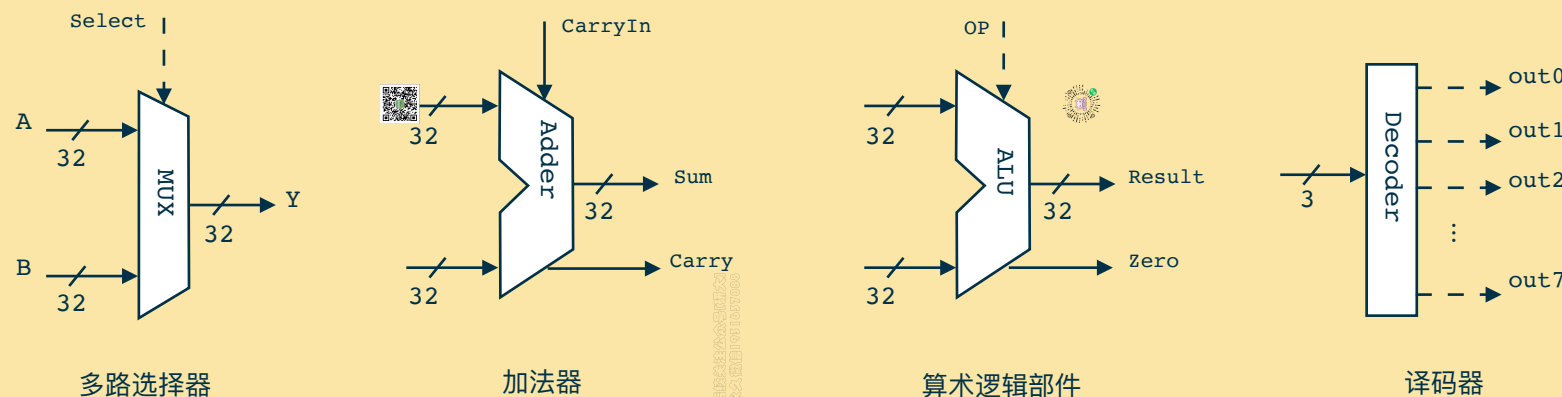




1. 数据通路的组成

1.1 组合逻辑元件



组合逻辑元件的**输出只取决于当前的输入**

组合电路的定时**不受时钟信号的控制**

所有输入信号到达后，**经过一定的逻辑门延迟**，输出端的值被改变，并**一直保持其值不变**，直到输入信号改变

常用的组合逻辑元件有**多路选择器 (MUX)**、**加法器 (Adder)**、**算术逻辑部件 (ALU)**、**译码器 (Decoder)** 等

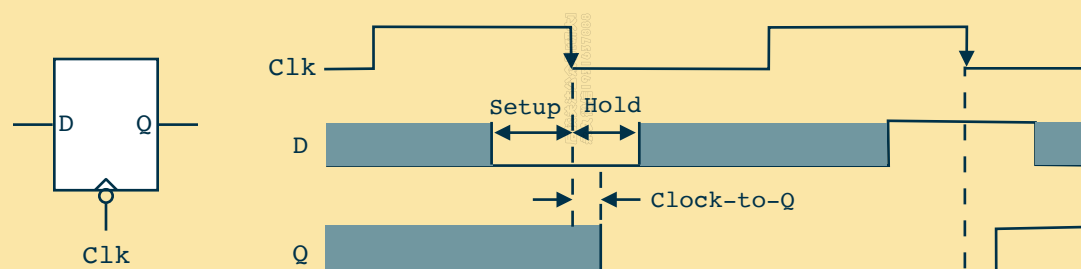


1. 数据通路的组成

1.2 状态元件

状态元件**具有存储功能**

输入状态在**时钟控制下**被写入电路中，并**保持电路的输出值不变**，直到下一个时钟到达
输入端状态由时钟决定何时被写入，输出端状态随时可以读出



D触发器定时示意图

D采用时钟下降沿触发，要使输出状态能正确地随着输入状态改变，必须满足以下时间约束：

- (1) 在时钟下降沿到来前的一定时间内，输入端D的状态必须稳定有效，这段时间为**建立时间 (Setup time)**
- (2) 在时钟下降沿到来后的一定时间内，输入端D的状态必须继续保持稳定不变，这段时间为**保持时间 (Hold time)**

在上述两个约束条件满足的情况下，经过时钟下降沿到来后的一段**延迟时间 (Clock-to-Q time)**
输出端Q的状态改变为输入端D的状态，并一直保持不变，直到下一个时钟到来



1. 数据通路的组成

1.2 状态元件



状态元件**具有存储功能**

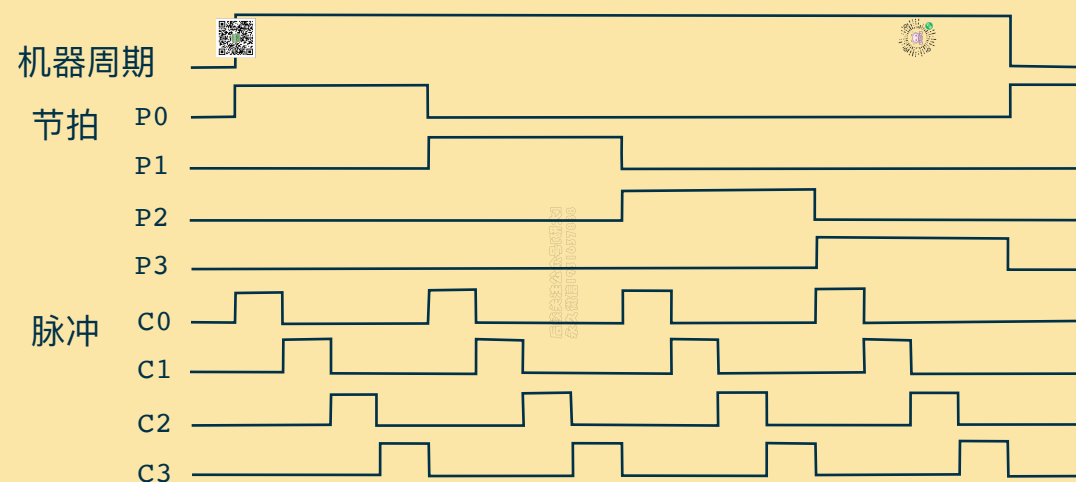
输入状态在**时钟控制下**被写到电路中，并**保持电路的输出值不变，直到下一个时钟到达**
输入端状态由时钟决定何时被写入，输出端状态随时可以读出

常用的状态元件有寄存器、暂存器、锁存器、IR、PC、通用寄存器组、
输出端带三态门的寄存器、带清零功能的寄存器、带自增功能的寄存器、带移位功能的寄存器等



2. 数据通路与时序控制

2.1 早期计算机的三级时序系统



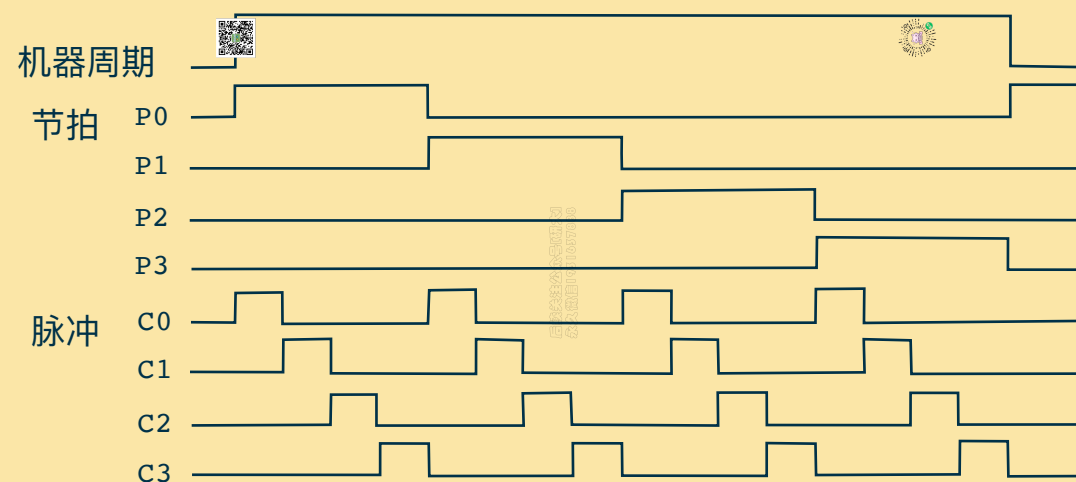
机器周期、节拍、脉冲三级时序系统

早期计算机通常采用**机器周期、节拍和脉冲三级时序**对数据通路操作进行定时控制



2. 数据通路与时序控制

2.1 早期计算机的三级时序系统



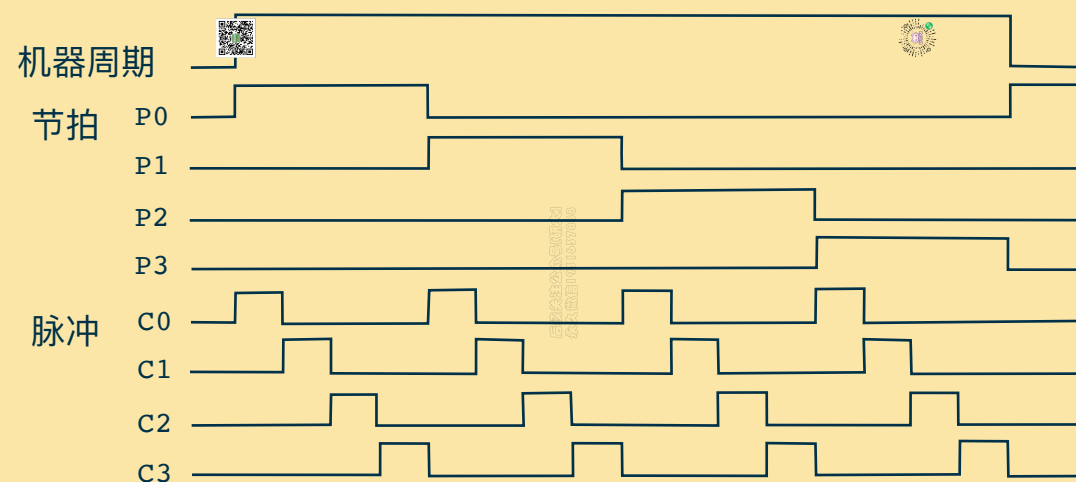
机器周期、节拍、脉冲三级时序系统

机器周期：一个指令周期可分为取指令、读操作数、执行并写结果等多个基本工作周期，即机器周期
每个机器周期长短可能不同，例如存储器读或写周期比CPU中的操作时间长得多
所以机器周期的宽度通常以主存工作周期为基础来确定



2. 数据通路与时序控制

2.1 早期计算机的三级时序系统



机器周期、节拍、脉冲三级时序系统

节拍和脉冲：一个节拍包含多个工作脉冲

一个机器周期内要进行若干步动作，例如存储器读周期有送地址、发读命令、检测数据有无准备好等

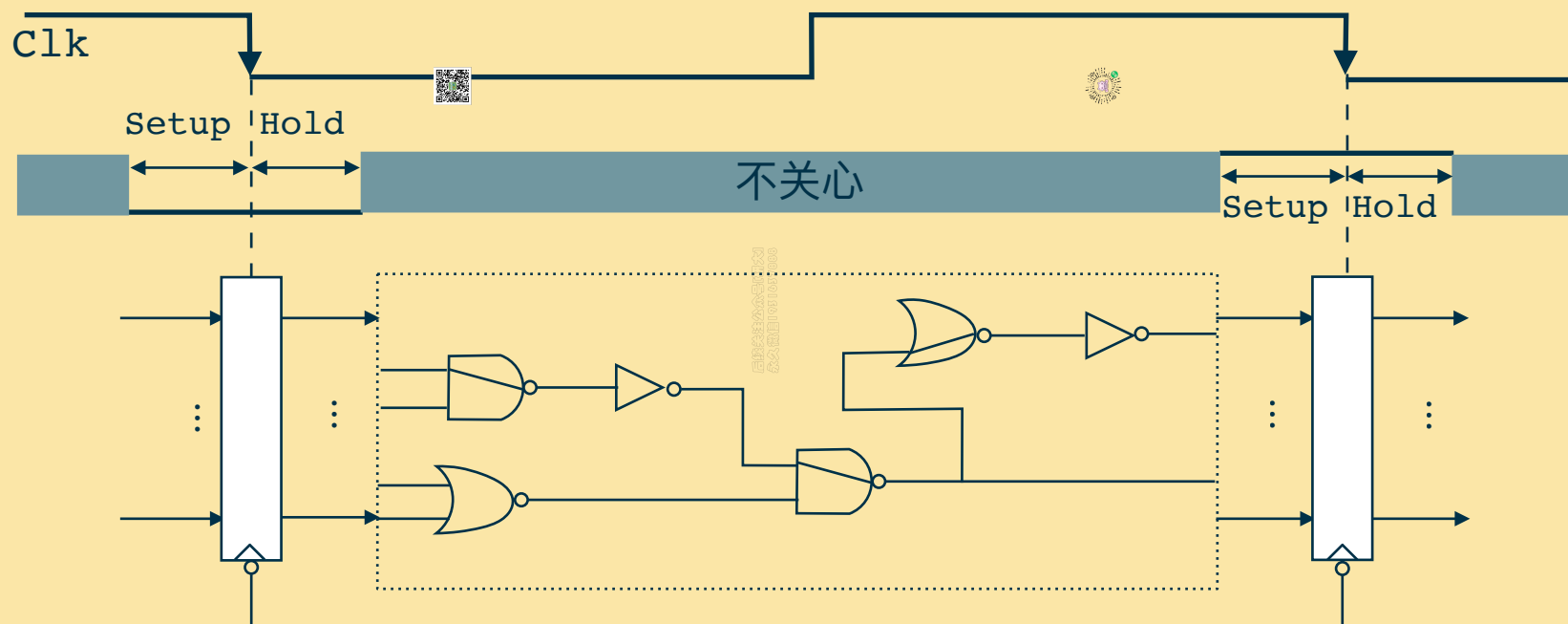
因此有必要将一个机器周期再划分成若干节拍，每个动作在一个节拍内完成

为了产生操作控制信号并使某些操作能在一拍时间内配合工作，常在一个节拍内再设置一个或多个工作脉冲



2. 数据通路与时序控制

2.2 现代计算机的时钟信号



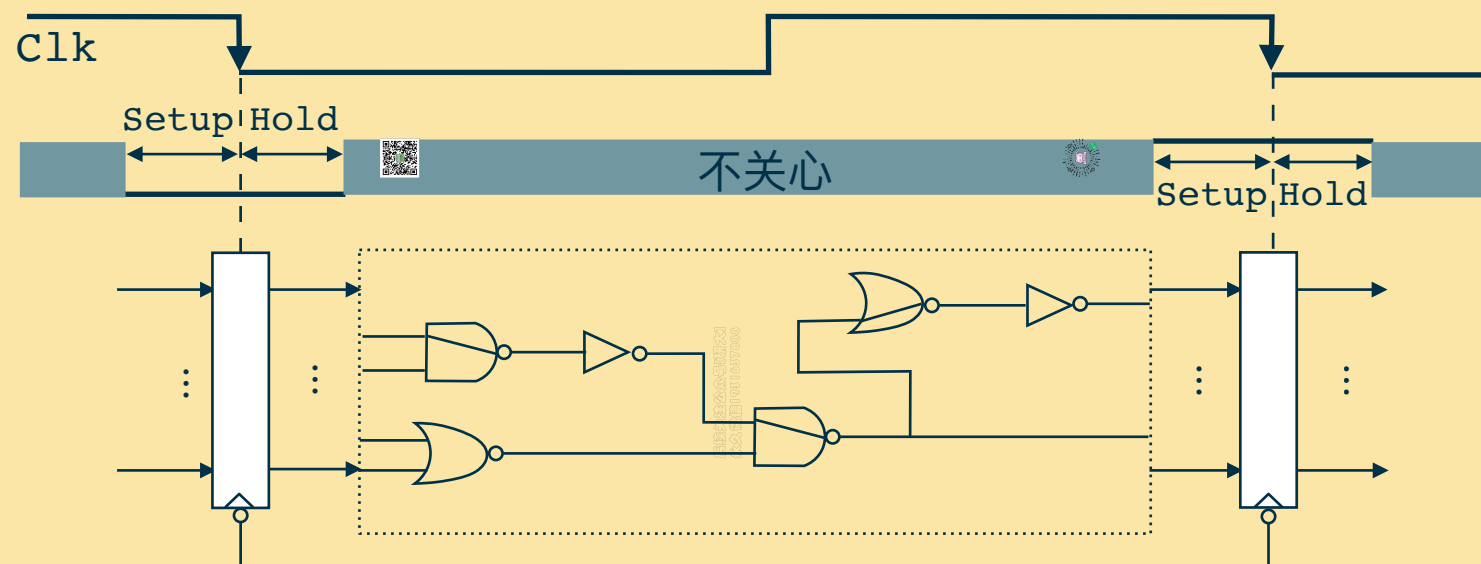
数据通路与时钟周期

现代计算机中已不再采用上述三级时序系统，机器周期的概念已逐渐消失
整个数据通路中的定时信号就是时钟，**一个时钟周期就是一个节拍**



2. 数据通路与时序控制

2.2 现代计算机的时钟信号



数据通路与时钟周期

CPU时钟周期

机器内部主时钟脉冲的宽度
是CPU工作的最小时间单位

时钟周期以相邻状态单元间组合逻辑电路的最大延迟时间为基准确定

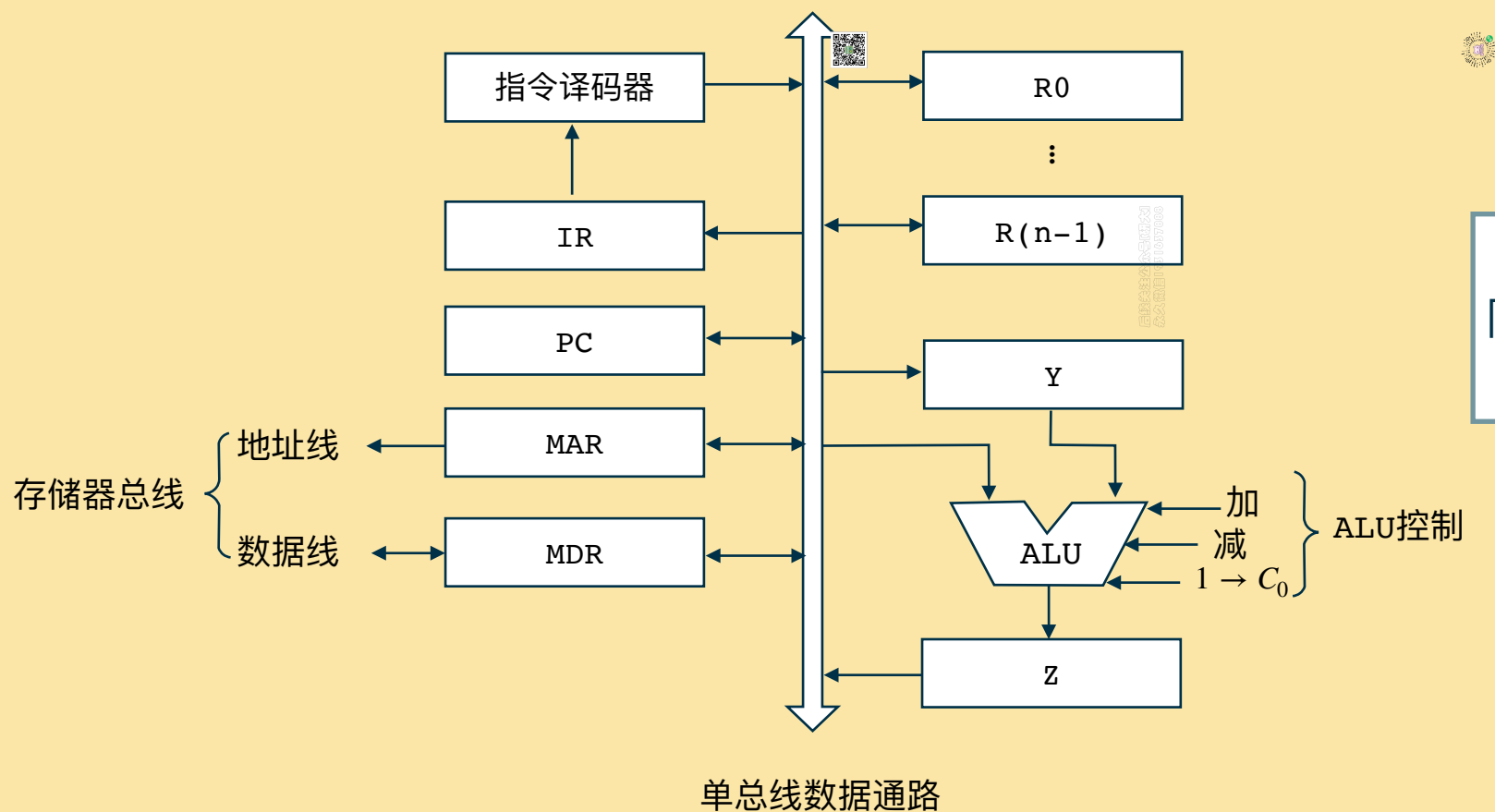
数据通路可看作由组合逻辑（操作）元件和状态（存储）元件交替组合而成

即数据通路的基本结构为“...-状态元件-操作元件（组合电路）-状态元件-...”



3. 总线式数据通路

3.1 单总线数据通路

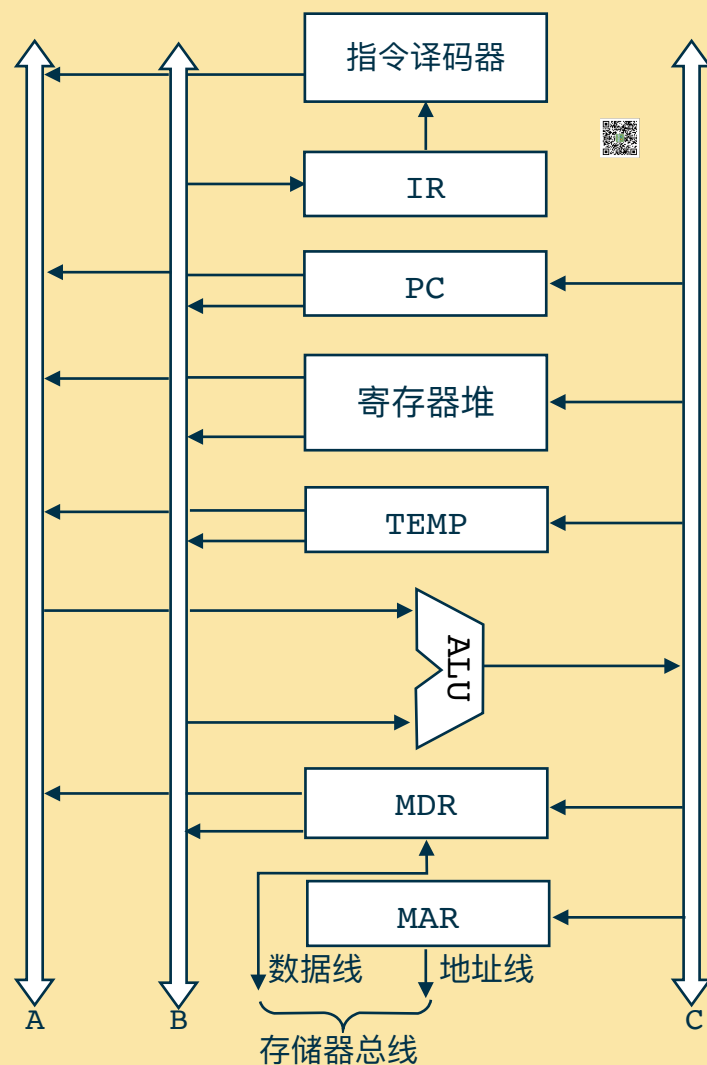


所有数据传输共享同一条内部总线
同一时刻仅允许一个部件向总线输出数据
因此一个时钟周期只允许传一个数据



3. 总线式数据通路

3.2 三总线数据通路



所有通用寄存器在一个双口寄存器堆中
允许两个寄存器的内容同时输出到A总线和B总线
和单总线结构相比，多总线在执行指令时所需的步骤大为减少



4. 单周期数据通路的设计

5条R-型指令



```
add rd, rs, rt  
sub rd, rs, rt  
subu rd, rs, rt  
slt rd, rs, rt  
sltu rd, rs, rt
```

5条I-型指令

```
ori rt, rs, imm16  
addiu rt, rs, imm16  
lw rt, rs, imm16  
sw rt, rs, imm16  
beq rs, rt, imm16
```

1条J-型指令

```
j target
```



4. 单周期数据通路的设计

5条R-型指令

指令	功能	说明
add rd,rs,rt sub rd,rs,rt	$M[PC], PC \leftarrow PC + 4$ $R[rd] \leftarrow R[rs] \pm R[rt]$	从PC所指的内存单元中取指令，并PC加4 从rs、rt中取数后相加减，若溢出则异常处理，否则结果送rd
subu rd,rs,rt	$R[rd] \leftarrow R[rs] - R[rt]$	从rs、rt中取数后相减，结果送rd（不进行溢出判断）
slt rd,rs,rt	if ($R[rs] < R[rt]$) $R[rd] \leftarrow 1$ else $R[rd] \leftarrow 0$	从rs、rt中取数后按带符号整数来判断两数大小 小于则rd中置1，否则，rd中清零 （不进行溢出判断）
sltu rd,rs,rt	if ($R[rs] < R[rt]$) $R[rd] \leftarrow 1$ else $R[rd] \leftarrow 0$	从rs、rt中取数后按无符号数来判断两数大小 小于则rd中置1，否则，rd中清零 （不进行溢出判断）



4. 单周期数据通路的设计

5条I-型指令

指令	功能	说明
<code>ori rt,rs,imm16</code>	$R[rt] \leftarrow R[rs] \mid \text{ZeroExt}(imm16)$	从rs取数，将imm16进行零扩展，然后两者按位或，结果送rt
<code>addiu rt,rs,imm16</code>	$R[rt] \leftarrow R[rs] \mid \text{SignExt}(imm16)$	从rs取数，将imm16进行符号扩展，然后两者相加，结果送rt（不进行溢出判断）
<code>lw rt,rs,imm16</code>	$\begin{aligned} \text{Addr} &\leftarrow R[rs] + \text{SignExt}(imm16) \\ R[rt] &\leftarrow M[\text{Addr}] \end{aligned}$	从rs取数，将imm16进行符号扩展，然后两者相加，结果作为访存地址Addr，从Addr中取数并送rt
<code>sw rt,rs,imm16</code>	$\begin{aligned} \text{Addr} &\leftarrow R[rs] + \text{SignExt}(imm16) \\ M[\text{Addr}] &\leftarrow R[rt] \end{aligned}$	从rs取数，将imm16进行符号扩展，然后两者相加，结果作为访存地址Addr，将rt送Addr中
<code>beq rs,rt,imm16</code>	$\begin{aligned} \text{Cond} &\leftarrow R[rs] - R[rt] \\ \text{if}(\text{Cond} \text{ eq } 0) \\ \text{PC} &\leftarrow \text{PC} + 4 + (\text{SignExt}(imm16) \times 4) \end{aligned}$	做减法以比较rs和rt中内容的大小，并计算下一条指令地址，然后根据比较结果修改PC。转移目标地址采用相对寻址，基准地址为下一条指令地址（即PC+4），位移量为立即数imm16经符号扩展后的值的4倍。因此，转移目标指令的范围相对于当前指令的前32767到后32768条指令



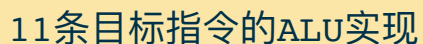
4. 单周期数据通路的设计

1条J-型指令



指令	功能	说明
j target	$PC<31:2>\leftarrow PC<31:28>\parallel target<25:0>$	第一步无须进行PC+4而直接计算目标地址，符号 $\parallel$ 表示“拼接”

4.1 算术逻辑部件的设计



- 加法 (add)
- 减法 (sub)
- 按位或 (or)
- 有符号比较 (slt)
- 无符号比较 (sltu)

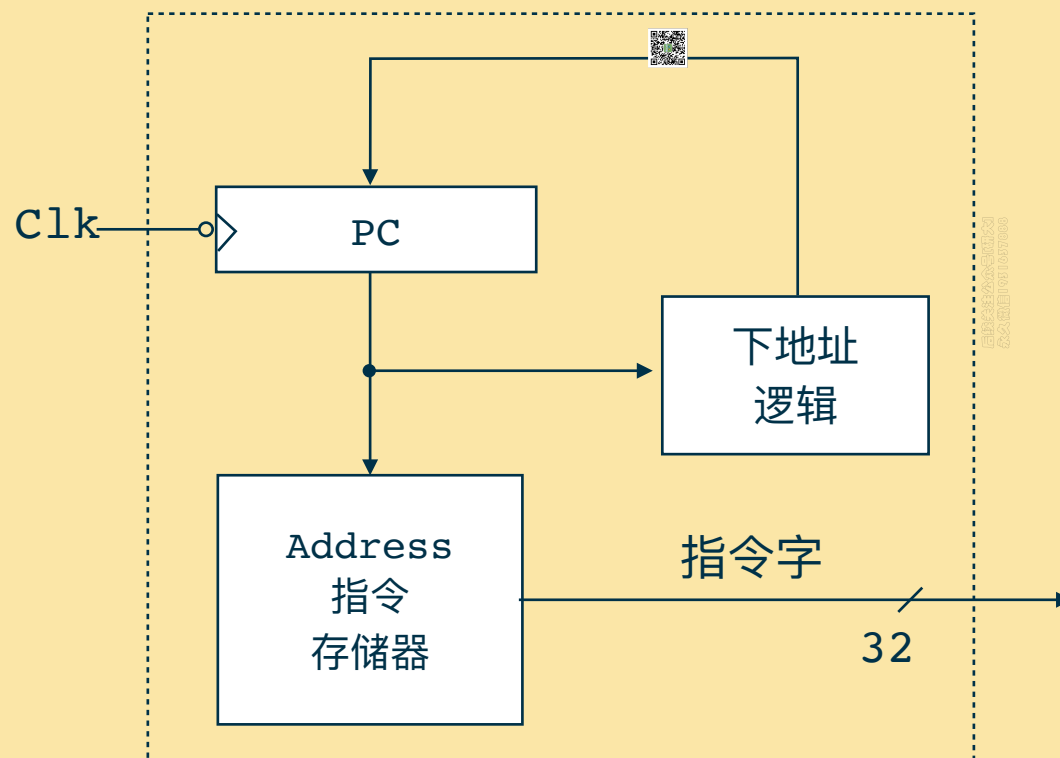
- Result (运算结果)
- Zero (结果是否为0)
- Overflow (是否发生溢出)

一个加法器+一套逻辑电路+多路选择器



4. 单周期数据通路的设计

4.2 取指令部件的设计



取指令部件示意图

功能

- 根据PC取出当前指令
- 计算下一条指令地址
- 更新PC

组成

- PC (程序计数器)
- 指令存储器
- 下地址逻辑

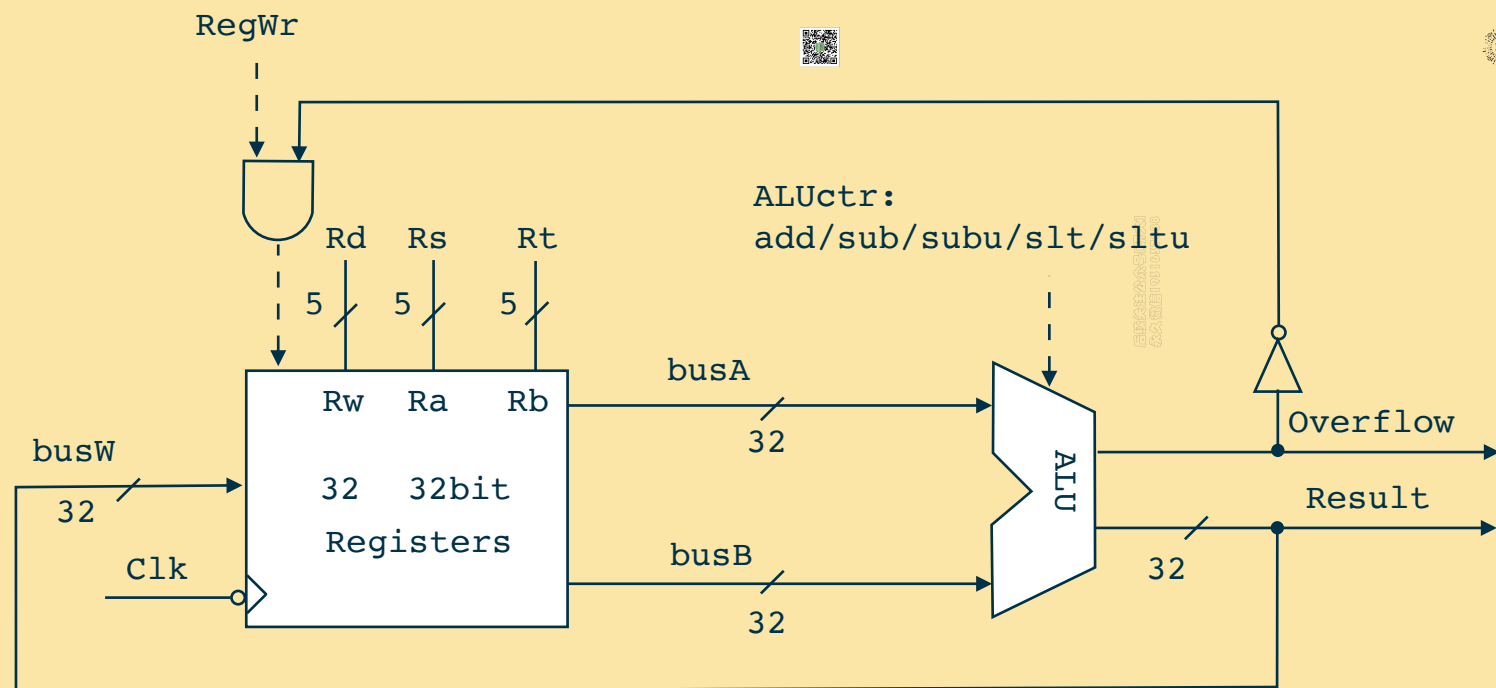
下地址的两种情况

- 顺序指令: $PC \leftarrow PC + 4$
- 转移执行: $PC \leftarrow \text{转移目标地址}$



4. 单周期数据通路的设计

4.3 R-型指令的数据通路



支持R-型指令功能的数据通路

执行流程

- 读取Rs, Rt
- ALU完成运算
- 结果写回Rd

主要控制信号

- RegWr: 是否写寄存器
- ALUctr: 选择ALU运算类型

4.4 立即数运算指令的数据通路



- **RegDst**多路选择器目的寄存器由rd变为rt
- **扩展器 (ExtOp)**：16位立即数需要扩展到32位
- **ALUSrc**多路选择器：ALU第二操作数改为可来自立即数

18

4.5 Load/Store指令的数据通路

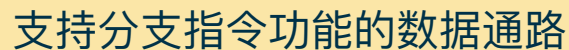


- **数据存储器**：增加数据存取功能
- **MemtoReg多路选择器**：写回数据来源改变

19

4.6 分支指令的数据通路

- **下地址逻辑**：增加条件分支功能
- **Branch控制逻辑**：根据比较结果决定是否转移



4.7 无条件转移指令的数据通路



- **Jump目标地址生成逻辑**：增加无条件跳转功能
- **Jump控制MUX**：下一条PC可选择Jump目标地址

完整的取指令部件

4.8 综合11条指令的完整单周期数据通路





真题训练

*此模块题目取自真题及与真题风格类似的模拟题



1. 下列有关处理器时钟信号的叙述中, 错误的是
- A. 时钟信号由机器脉冲源发出的脉冲信号经整形和分频后形成
 - B. 时钟信号的宽度称为时钟周期, 时钟周期的倒数为机器主频
 - C. 时钟周期以相邻状态单元间组合逻辑电路的最大延迟为基准确定
 - D. 处理器总是在每来一个时钟信号时就开始执行一条新的指令



真题训练

*此模块题目取自真题及与真题风格类似的模拟题



1. 下列有关处理器时钟信号的叙述中, 错误的是
- A. 时钟信号由机器脉冲源发出的脉冲信号经整形和分频后形成
 - B. 时钟信号的宽度称为时钟周期, 时钟周期的倒数为机器主频
 - C. 时钟周期以相邻状态单元间组合逻辑电路的最大延迟为基准确定
 - D. 处理器总是在每来一个时钟信号时就开始执行一条新的指令**